



Domande IS - Riassunto Ingegneria del Software e Lab.

Ingegneria del Software e Lab. (Università degli Studi di Modena e Reggio Emilia)



Scan to open on Studocu

DOMANDE PROVE D'ESAME

★Che cosa studia l'ingegneria del software?

Quali sono i problemi che tipicamente affronta questa materia?

[2 punti]

L'ingegneria del software è quella materia che nasce per ovviare ai diversi problemi emersi negli anni '60 dopo che i progetti software hanno acquisito sempre più complessità e dimensione

La materia principalmente affronta le problematiche legate alla progettazione di un software: dalla raccolta dei requisiti, fino al supporto a valle del rilascio del prodotto, comprendendo la gestione di tempi e costi, la manutenzione, la portabilità e la scalabilità del prodotto

Nasce dall'esigenza di imporre una disciplina formale a quella che fino a quel momento era pura "programmazione"

L'ingegneria del software stabilisce e usa metodologie solide per ottenere software economicamente affidabile e che lavori con macchine reali, è una forma di ingegneria che applica i principi della matematica per ottenere soluzioni cost effective a problemi software

L'obiettivo dell'ingegneria del software è la creazione di software di alta qualità in modo sistematico, controllato ed efficiente

★Quali sono i principali elementi in cui il processo di specifica dei requisiti può essere suddiviso e descritto? {Descrivere come può essere suddiviso il processo di specifica dei requisiti}

[3 punti]

L'ingegneria dei requisiti suddivide la specifica di ricerca dei requisiti in 7 fasi:

1. Elicitazione
2. Documentazione e definizione
3. Specifica
4. Prototipazione
5. Analisi
6. Revisione e validazione
7. Agreement e accettazione

★Cosa si intende per elicitation nella fase di analisi dei requisiti?

[3 punti]

L'elicitation dei requisiti è il processo di raccolta delle informazioni sul sistema da analizzare

Le persone coinvolte in questo processo sono note come stakeholder

L'elicitation coinvolge principalmente due figure: l'azienda, chiamata a produrre il software, e il cliente, il quale indica all'azienda le sue richieste

L'elicitation può essere eseguita a due livelli:

- ad alto livello: per capire la ragione di business e la giustificazione del software, l'azienda produttrice si interfaccia con la direzione aziendale (lato cliente), che valutando il mercato che il prodotto andrà ad occupare, esprime necessità e obiettivi che si aspetta dal prodotto stesso, la discussione non verte su aspetti tecnici
- a livello di dettaglio: per raccogliere le necessità e i desideri dell'utente, comporta l'interfacciarsi con l'utilizzatore del software, riguarda cosa il prodotto deve eseguire nello specifico (a livello tecnico)

- alcuni esempi: modalità di salvataggio di eventuali dati in memoria, user interface con cui l'utente dovrà interagire durante l'utilizzo del prodotto finale

***Quali sono le fasi in cui si suddivide il processo di analisi dei requisiti?**

[3 punti]

L'analisi dei requisiti consiste di due principali attività:

- Categorizzazione o clustering
- Prioritizzazione

La categorizzazione riguarda l'assegnazione di determinate categorie sulla base del requisito analizzato

Le categorie sono di diverso tipo (ad esempio: funzionalità singola, user interface, business workflow...), tuttavia tra esse non c'è una netta distinzione, infatti ci possono essere requisiti di mutua categoria dando vita a ciò che viene comunemente chiamato overlapping

La prioritizzazione consiste nella stima dei tempi secondo cui verranno processati e soddisfatti i requisiti

La priorità viene assegnata o in base al ruolo del requisito all'interno del software o in base al tempo necessario per implementare tale requisito

Esistono metodi per assegnare priorità come l'AHP, tecnica che dà al team una visione di quanto peserà, in termini di tempo e costo, un requisito rispetto ad un altro, fornendo una stima formale

***Che cosa si intende per SRS, a che cosa serve, in che fase del ciclo della produzione del software si colloca e quali sono le caratteristiche che rendono una SRS di qualità**

[3 punti]

SRS sta per software requirements specification, che indica un documento che descrive completamente il comportamento di un sistema da sviluppare

Dopo l'analisi e la revisione dei requisiti è prudente metterli nel documento

I dettagli che devono essere inclusi dipendono da diversi parametri:

- Dimensione e complessità del progetto
- Stimare il numero di clienti che supportano le attività
- Conoscenza ed esperienza degli sviluppatori nell'area di riferimento

Se il numero di clienti stimato è largo allora la specifica dei requisiti deve essere dettagliata e completa

Questo documento:

- Specifica i processi necessari implementati nelle attività, che risultano nei requisiti per il sistema e i prodotti software
- Fornisce linee guida per applicare i requisiti
- Specifica le informazioni necessarie prodotte durante l'implementazione dei requisiti di processo
- Specifica il contesto necessario per le informazioni
- Fornisce linee guida per il formato necessario per le informazioni

Il processo di raccolta deve produrre le seguenti informazioni:

- Specifica dei requisiti business
- Specifica dei requisiti degli stakeholder
- Specifica dei requisiti del sistema

- Specifica dei requisiti del software
(p. 48, pdf. 4)

***Che cosa indicano i principi di programmazione SOLID?**

[5 punti]

- The single responsibility principle: ogni componente del software deve avere una sola responsabilità, cioè un solo motivo per cambiare, quindi deve svolgere una sola funzionalità
 - si basa sul concetto di coesione, cioè su quanto un componente supporta un'unica responsabilità, ne consegue che nel design di un componente è richiesta un'alta coesione
 - il rispetto di questo principio comporta una migliore manutenibilità del codice, che può essere modificato o aggiornato con un minore impatto sul resto del sistema
- The open closed principle: il software deve essere aperto alle estensioni e chiuso rispetto alle modifiche
 - il codice già scritto e funzionante non deve essere modificato, nel caso si vogliano modificare o aggiungere delle funzionalità non si deve impattare sul codice esistente (si devono creare delle nuove classi)
 - alla base dei design pattern che favoriscono la composizione all'ereditarietà
- The liskov substitution principle: gli oggetti di una superclasse devono essere rimpiazzabili dagli oggetti di una sua sottoclasse senza rompere l'applicazione
 - se ho una classe B che eredita da una classe A, l'applicazione deve funzionare allo stesso modo sia con oggetti di tipo B che di tipo A
 - esempio: se ho un'interfaccia con diversi metodi e ho tante classi che svolgono una singola funzione, le classi sono costrette a implementare anche metodi al di fuori della singola responsabilità che svolgono
- The interface segregation principle: una classe deve dipendere solo dalle interfacce che sono per essa necessarie
 - i metodi di un'interfaccia devono essere coerenti con la sua funzionalità, non devono esserci metodi implementati che non riguardano la classe che li sta implementando
 - esempio: supponiamo di avere un'interfaccia *Atleta* e una classe *Calciatore* che eredita da *Atleta* i metodi *run()*, *jump()* e *swim()*; in questo caso il principio viene violato perché il calciatore non nuota, ma deve lo stesso implementare il metodo *swim()*
- The dependency inversion principle: dobbiamo dipendere da astrazioni e non da cose concrete
 - i moduli ad alto livello di astrazione non devono dipendere dai sottomoduli che ne forniscono un'implementazione
 - un cambiamento a basso livello non deve impattare sui moduli di alto livello

***Che cosa si intende per coesione e accoppiamento di moduli software**

Indicare se questa caratteristica è importante per la progettazione del software e come è possibile valutare il grado di accoppiamento e coesione di un software

[3 punti]

Uno dei nostri obiettivi durante la creazione di un software è quello di creare componenti

- fortemente coesi: cioè mettere insieme le cose che c'entrano le une con le altre
- poco accoppiati: cioè creare moduli poco dipendenti gli uni dagli altri

Per valutare il grado di accoppiamento e coesione di un software si utilizzano dei principi

I principi che regolano la coesione sono:

- Release equivalence principle (REP): la release mi determina i componenti che posso utilizzare
- Common closure principle (CCP): classi che cambiano per la stessa ragione e nello stesso momento vanno messe nella stessa release
- Common reuse principle (CRP): non devo forzare gli utenti di un componente a dipendere da cose che non usano

I principi che regolano l'accoppiamento sono:

- Acyclic dependencies principle (ADP): i vari pacchetti non devono essere in dipendenza ciclica
- Stable dependencies principle (SDP): devo dipendere in direzione della stabilità, cioè da qualcosa che varia meno velocemente di me
- Stable abstraction principle (SAP): più un componente deve essere astratto più al suo interno deve contenere classi astratte

★Indicare, anche attraverso esempi, cosa si intende per coesione di moduli software

[3 punti]

La coesione misura quanto fortemente ciascuna delle funzioni è correlata all'interno di un modulo. Se una certa classe sta eseguendo un insieme di funzioni altamente correlate, si dice che quella classe è coesa.

Coesione: caratteristica di un'unità che identifica il grado a cui gli elementi di questa unità appartengono.

In un'unità di design altamente coesa, i pezzi di questa unità di design sono tutti connessi tra loro per soddisfare un singolo obiettivo o una singola funzione.

Diversi livelli di coesione:

- Coesione casuale: l'unità di design o codice sta eseguendo tanti task non correlati, livello di coesione più basso
- Coesione logica: il design esegue una serie di task simili, gli elementi sono correlati ma le relazioni sono deboli
- Coesione temporale: mette insieme una serie di elementi correlati dal tempo, azioni collegate in termini di tempo, in una classe ho funzionalità accedute nello stesso momento nel software
- Coesione procedurale: coinvolge azioni correlate proceduralmente, azioni collegate in termini di sequenza
- Coesione comunicazionale: il design è correlato dalla sequenza delle attività, e le attività sono rivolte agli stessi dati
- Coesione sequenziale o funzionale: le unità di design eseguono una principale attività o raggiungono un obiettivo, una classe svolge una sola funzionalità

***Indicare in che modo gli use case diagram supportano il processo di realizzazione del software
A che livello vengono introdotti?**

Che tipo di informazione veicolano?

Fornire un esempio di use case diagram e spiegare con esso il significato delle relazioni “include” e “extend”

[4 punti]

Gli use case sono diagrammi utilizzati per raccogliere i requisiti funzionali di un sistema

Essi descrivono la tipica interazione tra gli utenti e il sistema o tra il sistema e sé stesso, fornendo una sorta di racconto di come il sistema è utilizzato

Uno use case diagram risponde alle seguenti domande:

- Cosa viene descritto? Il sistema
- Chi interagisce con il sistema? Gli attori
- Cosa possono fare gli attori? Le attività, quindi gli use case

Questo tipo di diagrammi vengono introdotti a livello dell'analisi dei requisiti e nel testing

Conoscere gli use case diagram ci permette di conoscere il dominio della nostra applicazione

L'operazione di inclusione indica una chiamata da parte di uno use case ad un altro use case

L'operazione di estensione indica l'introduzione di un caso particolare

***Spiegare che cosa si intende per architettura del software e fornire alcuni esempi di stili architetturali**

[3 punti]

L'architettura del software è la struttura del software stesso, cioè l'insieme degli elementi di alto livello di cui si compone il software, delle proprietà visibili all'interno del software, delle relazioni fra i componenti e fra componenti e proprietà

Le architetture possono essere:

- Monolitiche: il codice che gestisce tutte le funzionalità del progetto è su una sola macchina
- Distribuite: il progetto è gestito su più macchine in remoto che comunicano tra loro, architettura più elastica perché concepisce il software come funzionante su diversi dispositivi, i server possono essere specializzati a seconda dell'ambito in cui il componente deve essere utilizzato (ad esempio un server che deve gestire gli accessi di tanti utenti sarà più potente di un server che necessita di prestazioni più basiche), questo tipo di architettura è basata sulla rete (influenzata dal tempo di latenza e dalla stabilità della rete)

Degli esempi di stili architetturali possono essere:

- Pipes and filters
- Client server
- Layered
- Microkernel
- Event driven
- Microservizi
- Service based
- Serverless

Java propone una specifica libreria per utilizzare il design pattern observer

Spiegare il pattern e indicare in che modo e con quali varianti questa libreria implementa il pattern

[5 punti]

Il design pattern observer definisce una dipendenza uno a molti tra oggetti, in modo tale che quando un oggetto cambia di stato tutti gli oggetti da esso dipendenti sono notificati e aggiornati automaticamente

Il principio di design su cui si basa questo pattern è: favorire design che abbiano basso accoppiamento tra gli oggetti che interagiscono

Può essere implementato tramite due meccanismi:

- Push: quando si passano come parametri solo i dati necessari
- Pull: quando si passa come parametro tutto l'oggetto nella sua interezza

Java fornisce una specifica libreria per utilizzare l'observer che è `java.util.Observable`

Spiegare come è implementato in Java il design pattern Observer

[4 punti]

Il pattern specifica una comunicazione tra due tipologie di oggetti: gli *osservatori*, detti `Observer`, e gli *osservabili*, detti `Subject`

Un oggetto osservabile è quello che notifica agli osservatori i propri cambiamenti di stato

L'implementazione del pattern prevede che il `Subject` mantenga una lista di tutti gli `Observer` registrati

A tale scopo espone un metodo per la registrazione ed uno per la cancellazione degli osservatori

Questi ultimi, invece, espongono un metodo di aggiornamento, `update()`, che sarà invocato dall'oggetto osservato ad ogni cambiamento del proprio stato

Tale evento è rappresentato nell'implementazione proposta dal metodo `notifyObservers()`

*Che cos'è il principio di "Least Knowledge" (principio della conoscenza minima o legge di Demetra)?

Indicare un pattern in cui questo principio viene utilizzato

[4 punti]

Ogni unità di programma dovrebbe conoscere solo poche altre unità di programma strettamente correlate

Ogni unità di programma dovrebbe interagire solo con le unità che conosce direttamente.

Questi due principi possono essere riassunti col motto "non parlate con gli sconosciuti"

In programmazione a oggetti, in particolare, questo implica che un oggetto non dovrebbe interagire direttamente con (usare operazioni di) oggetti a cui accede solo indirettamente (attraverso operazioni o attributi dei suoi conoscenti diretti)

Questo garantisce che un oggetto sia indipendente dalla struttura interna e dalle proprietà degli altri oggetti, compresi i loro eventuali componenti interni o le loro relazioni

Il facade pattern utilizza il principio di Demetra e in particolare, il suo utilizzo, comporta

- Fare chiamate più corte
- Chiamare metodi che dipendono da:
 - L'oggetto stesso
 - Oggetti passati come parametri al metodo

- Oggetti che il metodo crea o istanzia
- Componenti dell'oggetto

Un principio di design del software indica di favorire architetture che usano composizione rispetto a quelle che usano ereditarietà

Per quale motivo?

Utilizzare qualche esempio per descrivere il concetto

[4 punti]

È consigliato prediligere l'uso della composizione piuttosto che dell'ereditarietà perché l'implementazione tramite composizione favorisce l'astrazione, infatti viene attuata tramite la creazione di un sistema di interfacce

Questo principio viene utilizzato ad esempio dallo strategy

Se dobbiamo creare un personaggio con dei comportamenti, invece di fargli ereditare i comportamenti, favoriamo la composizione e cioè facciamo in modo che il personaggio abbia i suoi comportamenti composti con oggetti comportamento

***Indicare che cosa si intende per analisi statica del software, e illustrare i vantaggi di questo tipo di analisi**

[4 punti]

L'analisi statica è il processo di valutazione di un sistema o di un suo componente basato sulla sua forma, sulla sua struttura, sul suo contenuto o sulla documentazione di riferimento

Ciò significa che la valutazione avviene a prescindere dall'esecuzione del sistema

(Esempio: lines of code, nested level...)

L'analisi dinamica è il processo di valutazione di un sistema software o di un suo componente basato sull'osservazione del suo comportamento in esecuzione

(Esempio: code coverage, bugs per line of code...)

***È importante utilizzare delle metriche per qualificare il codice di un software?**

A cosa servono le metriche?

Proporre esempi di metriche

[3 punti]

{Che cosa si intende per software metrics?

È sempre vantaggioso definire e utilizzare delle misure quantitative che descrivono il software?

Introdurre una misura che si considera particolarmente interessante

[4 punti]}

Una metrica del software rappresenta una misura quantitativa del grado di cui un sistema software, un componente o un processo possiede una definita proprietà o caratteristica

L'obiettivo delle metriche del software è di fare valutazioni durante tutto il ciclo di vita del software per confermare se la qualità del software incontra o meno i requisiti

L'uso delle metriche del software riduce la soggettività nella valutazione e nel controllo della qualità del software fornendo basi quantitative per prendere decisioni sulla qualità del software

Tuttavia l'uso delle metriche del software non elimina la necessità del giudizio umano nella valutazione del software

L'uso delle metriche del software in un progetto dovrebbe avere un effetto benefico rendendo la qualità del software più visibile

I limiti nell'utilizzo delle metriche sono nel non poter stabilire dei range universali in cui il software rientra, rischiando di adattare il proprio prodotto per soddisfare le metriche stesse

Tipi di metriche che possono essere utilizzati:

- Metriche di prodotto: caratterizzano un prodotto intermedio o finale (esempio: dimensione del codice, complessità...)
- Metriche di processo: caratterizzano metodi, tecniche e strumenti utilizzati per sviluppare, implementare e mantenere il software (esempio: manutenibilità, velocità di risoluzione di bug...)
- Metriche di progetto: caratterizzano il progetto i termini di tempo, costo e management (esempio: tempistiche di progetto, produttività...)

Una metrica del software rappresenta una misura quantitativa del grado di cui un sistema software, un componente o un processo possiede una definita proprietà o caratteristica

Esempi:

- Lines of code: metriche che misurano quantitativamente la dimensione di una sorgente di codice contando le linee di codice
- Nesting level: metriche che misurano quantitativamente la complessità di una sorgente di codice misurando la profondità dell'innestamento delle strutture
- Halstead complexity:

★È opportuno commentare il codice?

In che modo dovrebbero essere scritti i commenti?

[2 punti]

I commenti possono aiutare o ledere la leggibilità e la manutenibilità del codice

Essi possono: distogliere l'attenzione dal codice, rendere il programma più difficile da leggere, essere errati, inoltre devono essere aggiornati ogni volta che viene aggiornato il codice

Ci sono 6 tipi di commenti diversi:

- Ripetizione del codice: commenti su istruzioni banali, questi commenti devono essere evitati perché sono inutili
- Spiegazione del codice: commento che spiega quello che è stato scritto tramite il codice, il codice dovrebbe essere comprensibile senza spiegazione, se così non fosse allora dovrebbe essere riscritto
- Indicatori nel codice: indicazioni sotto forma di commento che servono per capire cosa fare in futuro o cosa ricontrollare, indicano oggetti incompleti, miglioramenti, tengono traccia dei cambiamenti (commenti corretti)
- Riassunto del codice: commenti che riassumono ciò che fa il codice, sono utili per capire il codice ma vanno tenuti aggiornati (commento corretto, ma evitabile)
- Descrivere l'intenzione del codice: descrivono cosa il codice dovrebbe fare e non cosa fa, se seguiamo l'open closed principle allora l'intenzione del codice non cambierà mai (quindi commento corretto)
- Riferimenti esterni: commenti che collegano il codice a entità esterne, mettere riferimenti a quello che abbiamo fatto ("Ho implementato l'algoritmo X")

★È ritenuta una buona norma spiegare con commenti dettagliati che operazione compie una specifica linea di codice complessa?

[3 punti]

Non è ritenuta una buona norma perché se il codice fosse così complesso da essere incomprensibile dovrebbe essere riscritto invece di inserire un commento che lo spiega

***Che cosa è il debugging?**

In quali fasi si scompone?

[4 punti]

Il debugging è l'atto di localizzare e correggere gli errori nel codice, errori che sono stati precedentemente intercettati tramite il testing

Il debugging è un processo iterativo in cui: si crea un'ipotesi sulla causa dell'errore, si scrivono dei test per provare l'ipotesi e poi si cambia il codice per provare a risolvere il problema

Il debugging si divide in 4 fasi:

- Stabilizzazione o riproduzione: si riproduce l'errore su una configurazione per capire le condizioni che ci hanno condotto all'errore costruendo un test case
- Localizzazione: si trova la sezione di codice che provoca l'errore
- Correzione: si cambia il codice per risolvere l'errore
- Verifica: si verifica che gli errori siano stati corretti e che non siano stati introdotti nuovi errori nel codice

***Che cosa si intende per refactoring del codice?**

È considerato essere una buona pratica di implementazione?

Per quale motivo?

In che cosa consiste?

Ci sono rischi connessi con questa operazione?

[3 punti]

Il refactoring consiste nell'attività di miglioramento dello stile del codice senza alterarne il comportamento

Il codice può essere migliorato se:

- c'è codice duplicato
- i metodi sono lunghi
- le classi sono grandi
- abbiamo molti switch
- un metodo tende ad usare un oggetto di una classe diversa da quella da cui proviene
- una classe si riferisce troppo a parti private di altre classi

Il refactoring è un cambiamento nella struttura interna del software che lo rende più facile da capire e da modificare senza cambiarne il comportamento

Tipi di refactoring:

- extract method: processo che trasforma un frammento di codice in un proprio metodo
- substitute algorithm: un processo che sostituisce il corpo di un metodo con un nuovo algoritmo più chiaro e ritorna lo stesso risultato
- move method: un processo che sposta un algoritmo da una classe all'altra in cui ha più senso
- extract class: un processo che divide una classe in più classi

***Indicare gli aspetti più importanti che riguardano la fase di implementazione del software**
È considerata una buona norma riscrivere più volte il codice?

L'implementazione è la fase che trasforma i requisiti e i concetti del design in codice con un determinato linguaggio di programmazione

Le caratteristiche dell'implementazione sono:

- Leggibilità: quanto il codice è comprensibile per chi lo legge (poco innestato, pochi commenti)
- Manutenibilità: facilmente modificabile
- Performance: il codice deve lavorare il più veloce possibile
- Tracciabilità: il codice deve seguire un certo elemento di progettazione o design
- Correttezza: l'implementazione svolge la funzione per cui è nato il software (valutazione e validazione)
- Completezza: tutti i requisiti sono soddisfatti

Riscrivere più volte il codice non è considerata una buona norma perché spesso conduce all'introduzione di errori, in particolare se viene copiato un codice con errori essi vengono propagati all'interno di tutta l'applicazione e possono essere difficili da scovare e gestire
Il codice copiato o duplicato rende il software difficile da mantenere

Viene sempre consigliato di scrivere il codice una volta sola

Per gestire la duplicazione del codice però si possono utilizzare delle tecniche di refactoring quali: extract method e extract superclass

*Indicare le differenze tra black e white test

L'analisi degli use case diagram è utile per scrivere dei white test efficaci?

[3 punti]

Il black box testing è una tecnica di testing che non ha conoscenza del lavoro interno all'applicazione

Il programmatore simula di essere un utente, non guardiamo il codice, ma l'esecuzione dell'applicazione

Analisi del software senza consultare il codice

Vantaggio principale: non necessità di essere attuato da programmatori esperti

Svantaggi: la copertura è limitata, non riusciamo a tenere traccia delle aree di codice che testiamo

Il white box testing è una tecnica che fornisce un controllo dettagliato della logica interna e della struttura del codice

Il tester necessita di guardare il codice sorgente e trovare che unità di codice si sta comportando in modo inappropriato

Analisi del software con consultazione dell'implementazione del codice

Vantaggi: massima copertura del codice

Svantaggi: necessita di figure professionali esperte e di strumenti specifici, difficile da mantenere

Gli use case diagram rappresentano i requisiti dell'applicazione che stiamo testando, quindi ci indicano cosa deve fare l'applicazione, questo è proprio l'obiettivo del black box testing

Modalità per assicurarci che abbiamo testato scenari tipici e eccezionali per il sistema, dal punto di vista dei vari attori che interagiscono direttamente con il sistema e dal punto di vista dei vari stakeholder che interagiscono indirettamente con il sistema

Gli use case rappresentano i requisiti quindi conoscendo questo diagramma possiamo identificare il domino del sistema

Gli use case rappresentano sia i casi validi che i casi particolari (extend)

★Indicare cosa si intende per white box test e in che modo/che cosa verificano le principali tecniche di white box testing

[4 punti]

{Indicare le principali caratteristiche del white box test fornendo opportuni esempi di tale tecnica

[5 punti]

}

Controllo dettagliato della logica interna e della struttura del codice

Il tester necessita di guardare il codice sorgente e trovare che unità di codice si sta comportando in modo inappropriato

Analisi del software con consultazione dell'implementazione del codice

Vantaggi:

- il tester conosce il codice sorgente quindi è facile identificare che tipi di dati aiutano nel testing
- aiuta a ottimizzare il codice
- linee di codice extra possono essere rimosse, il che può portare a difetti nascosti
- massima copertura del codice perchè il tester conosce il codice

Svantaggi:

- il tester deve essere specializzato quindi i costi aumentano
- a volte è impossibile guardare ogni angolo del codice per trovare errori nascosti
- difficile da mantenere perchè necessita di strumenti specifici

Tecniche di white box testing:

Statement coverage

Tecnica che valuta le linee di codice

Tecnica di controllo di flusso meno efficiente

Minimo livello di copertura che dovrebbe essere accettabile

Se non riusciamo a testare tutto il codice e dobbiamo lasciare indietro qualcosa, conviene testare i percorsi meno probabili, perché quelli più probabili verranno testati quasi sicuramente da qualcun altro

Gli errori logici sono inversamente proporzionali alla probabilità dell'esecuzione di un percorso

Quando utilizziamo lo statement coverage dobbiamo stare attenti perché a volte con un coverage del 100% (cioè abbiamo testato tutte le linee di codice) non riusciamo effettivamente a testare tutti i casi possibili

Decision coverage

Tecnica che valuta le decisioni, ogni decisione ha la possibilità di essere risolta come vera o falsa

Se ho un decision coverage del 100% ho anche statement coverage del 100%

Loop coverage

Tecnica che testa i cicli

Per testare un ciclo si possono prendere come riferimento i seguenti valori:

- un valore che è 1 meno il minimo valore che il ciclo può prendere
- minimo numero di iterazioni
- 1 più del valore minimo
- Fare il ciclo una volta
- Fare il ciclo 2 volte
- Valore nominale, cioè tipico

- 1 meno del valore massimo
- Massimo numero di valori
- 1 più del valore massimo

Condition coverage

Tecnica che valuta le decisioni non atomiche, quindi le decisioni composte da condizioni

Testo il caso true e il caso false per ogni condizione

Una condizione atomica non può essere decomposta e non può contenere condizioni legate da operatori logici

Il condition coverage non implica il decision coverage, quindi facendo il primo potremmo non rispettare il secondo

Modified condition / decision coverage

Tecnica che guarda l'impatto della condizione nella decisione

Ogni condizione atomica deve essere testata in entrambi i modi e il decision coverage deve essere rispettato

Per ogni condizione atomica c'è almeno un test case in cui l'intero predicato viene valutato come falso solo perché quella specifica condizione atomica è falsa

Per ogni condizione atomica c'è almeno un test case in cui l'intero predicato viene valutato come vero solo perché quella specifica condizione atomica è vera

(esempio: la decisione deve essere vera perché A è vera, a causa di A)

Utile per chi sta testando software critici, ma non esaustivo in qualsiasi situazione

Un valore neutrale per una condizione atomica è un valore che non ha effetto sul risultato dell'espressione, ad esempio per l'and il valore neutrale è il true, mentre per l'or è il false

Garantisce un alto livello di sicurezza

Ci sono attività che una software house dovrebbe avere la necessità di pianificare dopo il rilascio del software, oppure con il rilascio del software si ritiene generalmente concluso un progetto di sviluppo di un software?

[3 punti]

Una software house dopo il rilascio di un software deve garantire la manutenzione dello stesso fornendo assistenza ai clienti qual ora essi trovassero errori o avessero problemi nell'utilizzo del prodotto rilasciato

Questo implica fare formazione riguardo al software al cliente e creare un team che deve includere: degli esperti in grado di modificare al momento del bisogno il codice del software; degli addetti alla comunicazione tra cliente e software house i quali devono rispondere alle chiamate del cliente e cercare di risolvere il problema visionando la documentazione e le FAQ fornite dalla software house, se il problema non è risolvibile in questo modo viene contattato il team tecnico (parte del team composta dagli esperti)

Un software dopo essere sviluppato va lentamente verso la sua caduta fino ad essere sostituito, ma questo processo può durare molti anni, in questi anni è fondamentale aggiornare e mantenere il software, garantire questo tipo di assistenza ci assicura anche un riscontro economico positivo

DOMANDE POSSIBILI ESAME

Introduction

***Quali sono le principali differenze nelle esigenze che intercorrono quando si programma un software di piccole dimensioni realizzato per soddisfare una esigenza limitata e un software complesso?**

La realizzazione di un programma semplice non richiede particolari accortezze

I problemi nascono quando il prodotto inizia ad essere più complesso, poiché la complessità implica maggiore attenzione nella progettazione del software, nel processo e nella manutenzione del codice

Un prodotto complesso deve essere realizzato da un gruppo di persone, le quali devono collaborare e definire una metodologia attraverso cui realizzare i desideri del cliente

Per risolvere problemi di grandi dimensioni usiamo il metodo dividi e conquista, quindi concettualizziamo l'applicazione dividendola in blocchi tramite diverse tipologie di decomposizione (tra cui: decomposizione funzionale, decomposizione rispetto agli utenti, decomposizione per astrazione)

Le idee chiave della divisione del software sono:

- Decomposizione, attraverso la quale si trovano le funzioni del software
- Modularizzazione, attraverso la quale si raggruppano le funzioni in moduli
- Separazione, attraverso la quale si isola il codice per non propagare errori

Più parzializziamo il lavoro più abbiamo bisogno di una metodologia da seguire

Dopo che il prodotto è stato suddiviso in componenti, questi ultimi vengono assegnati ai vari membri del team

***Che cosa si intende per ingegneria del software e quali sono i problemi che questa disciplina si propone di affrontare?**

L'ingegneria del software è quella materia che nasce per ovviare ai diversi problemi emersi negli anni '60 dopo che i progetti software hanno acquisito sempre più complessità e dimensione

La materia principalmente affronta le problematiche legate alla progettazione di un software: dalla raccolta dei requisiti, fino al supporto a valle del rilascio del prodotto, comprendendo la gestione di tempi e costi, la manutenzione, la portabilità e la scalabilità del prodotto

Nasce dall'esigenza di imporre una disciplina formale a quella che fino a quel momento era pura "programmazione"

L'ingegneria del software stabilisce e usa metodologie solide per ottenere software

economicamente affidabile e che lavori con macchine reali, è una forma di ingegneria che applica i principi della matematica per ottenere soluzioni cost effective a problemi software

L'obiettivo dell'ingegneria del software è la creazione di software di alta qualità in modo sistematico, controllato ed efficiente

***Che cosa implica la produzione di un software e qual è l'output generalmente atteso da questo processo?**

La produzione di un software implica una richiesta da parte di un cliente che deve soddisfare una particolare esigenza applicativa, per fare ciò bisogna passare attraverso diversi processi quali la raccolta dei requisiti, la gestione dei tempi di lavoro e l'implementazione

Durante il deployment, il cliente comincerà ad utilizzare il prodotto e a questo punto emergeranno eventuali ulteriori richieste da parte del cliente stesso, così come errori o bug del prodotto

Requirements Engineering

***Quali sono le principali fasi in cui il processo di specifica dei requisiti può essere suddiviso?**

L'ingegneria dei requisiti suddivide la specifica di ricerca dei requisiti in 7 fasi:

1. Elicitazione
2. Documentazione e definizione
3. Specifica
4. Prototipazione
5. Analisi
6. Revisione e validazione
7. Agreement e accettazione

***Qual è la differenza tra l'operazione di elicitation eseguita ad alto livello / a livello di dettaglio?**

L'elicitation coinvolge principalmente due figure: l'azienda, chiamata a produrre il software, e il cliente, il quale indica all'azienda le sue richieste

L'elicitation può essere eseguita a due livelli:

- ad alto livello: per capire la ragione di business e la giustificazione del software, l'azienda produttrice si interfaccia con la direzione aziendale (lato cliente), che valutando il mercato che il prodotto andrà ad occupare, esprime necessità e obiettivi che si aspetta dal prodotto stesso, la discussione non verte su aspetti tecnici
- a livello di dettaglio: per raccogliere le necessità e i desideri dell'utente, comporta l'interfacciarsi con l'utilizzatore del software, riguarda cosa il prodotto deve eseguire nello specifico (a livello tecnico)
 - alcuni esempi: modalità di salvataggio di eventuali dati in memoria, user interface con cui l'utente dovrà interagire durante l'utilizzo del prodotto finale

***Indicare le principali caratteristiche dell'analisi dei requisiti di alto livello e di dettaglio**

L'analisi dei requisiti consiste di due principali attività:

- Categorizzazione o clustering
- Prioritizzazione

La categorizzazione riguarda l'assegnazione di determinate categorie sulla base del requisito analizzato

Le categorie sono di diverso tipo (ad esempio: funzionalità singola, user interface, business workflow...), tuttavia tra esse non c'è una netta distinzione, infatti ci possono essere requisiti di mutua categoria dando vita a ciò che viene comunemente chiamato overlapping

La prioritizzazione consiste nella stima dei tempi secondo cui verranno processati e soddisfatti i requisiti

La priorità viene assegnata o in base al ruolo del requisito all'interno del software o in base al tempo necessario per implementare tale requisito

Esistono metodi per assegnare priorità come l'AHP, tecnica che dà al team una visione di quanto peserà, in termini di tempo e costo, un requisito rispetto ad un altro, fornendo una stima formale

***Cosa si intende per analisi dei requisiti e in che modo differisce dall'operazione di elicitation?**

L'elicitazione dei requisiti è il processo di raccolta delle informazioni sul sistema da analizzare
Le persone coinvolte in questo processo sono note come stakeholder

L'analisi dei requisiti è il processo di definizione delle aspettative degli stakeholder riguardanti i prodotti di un determinato progetto

Mira a comprendere i requisiti dichiarati dagli stakeholder (dopo che essi sono stati raccolti durante la fase di elicitazione)

***La prototipazione dei requisiti include anche la realizzazione di diagrammi UML di tipo Use Case
Che cosa si vuole rappresentare con questo tipo di diagrammi?**

Quali sono le informazioni che rappresentano questo tipo di diagrammi?

I diagrammi UML aiutano la stesura dei requisiti del sistema, i quali sono descritti, analizzati e racchiusi schematicamente per una comprensione rapida ed efficace del progetto intero

In particolare, lo use case è utilizzato per rappresentare i requisiti funzionali (descrivono le funzionalità e i servizi offerti dal sistema) di un sistema e come un utente interagisce con il sistema stesso, quindi può descrivere le funzionalità, il flusso degli eventi oppure le fasi in cui si può manifestare un errore durante l'utilizzo del software

Racchiude elicitazione e analisi in un unico grafico

Le informazioni rappresentate nello use case sono:

- Cosa viene descritto? Il sistema
- Chi interagisce con il sistema? Gli attori
- Cosa possono fare gli attori? Le attività, cioè gli use case

***Quali sono gli standard con cui si rappresentano i requisiti?**

Seguendo la norma di riferimento bisogna rappresentare i requisiti in un documento che racchiude le seguenti informazioni

- BRS (Business Requirements Specifications)
- StRS (Stakeholder Requirements Specifications)
- SyRS (System Requirements Specifications)
- SRS (Software Requirements Specifications)

Design Pattern

Quali sono i principi di programmazione che motivano la realizzazione e l'uso dei design pattern?

I principi di programmazione che motivano la realizzazione e l'uso di design pattern sono:

- Incapsulamento: nascondere i dati dietro il codice che li manipola
- Ereditarietà: un oggetto acquisisce le proprietà di un altro oggetto
- Polimorfismo: processare oggetti diversi basandosi sul loro tipo

Per ogni pattern introdotto a lezione (Strategy, Observer, Decorator, Factory, Singleton, Command, Adapter, Facade, Template, Iterator, Composite, State, Proxy, Builder, Bridge, Chain of Responsibility) può essere chiesto:

- Motivazione
- Uso
- Funzionamento
- Esempio rilevante
- Pseudo codice
- Applicazione a un determinato ambito
- Class diagram
- In che modo il diagramma soddisfa in principio di programmazione

Pdf "Riassunto design pattern"

Dato un principio di design viene chiesto di indicare un pattern (o quale pattern) implementa quel particolare principio di design

Principio di design	Design pattern
Identifica gli aspetti della tua applicazione che variano e separali da quelli che rimangono sempre gli stessi (SRP)	Strategy
	Factory
	Iterator
Occorre programmare verso un'astrazione, non verso un'implementazione concreta (DIP)	Strategy
	Factory method
Favorire la composizione all'ereditarietà	Strategy
Favorire design che abbiano basso accoppiamento tra gli oggetti che interagiscono	Observer
Le classi devono essere aperte all'estensione e chiuse alle modifiche (OCP)	Decorator
Principio di Demetra: dobbiamo conoscere il minimo di una classe, parla solo con i tuoi amici più vicini	Facade
Hollywood principle: non chiamarci, ti chiamiamo noi	Template

Differenza tra simple factory e static factory

Con l'Abstract Factory una classe delega la responsabilità dell'istanziamento di un oggetto a un altro oggetto tramite la composizione

Il Factory Method usa l'ereditarietà e affida la gestione dell'istanziamento dell'oggetto desiderato a una sottoclasse

Architecture and Methodology

*Cosa si intende per design di alto livello / di dettaglio?

Nella realizzazione del design di un progetto software ci sono due fasi principali:

- Fase di design architetturale: in cui si stipula una lista delle funzioni, dei componenti e delle relazioni tra i componenti stessi, a livello generale, e si raccolgono i requisiti
- Fase di design di dettaglio: i componenti vengono scomposti in più livelli e descritti in modo più tecnico, si lavora all'implementazione a livello di codice dei requisiti raccolti nella prima fase

*Quali sono i principi di programmazione SOLID? Fornire esempi

- **The single responsibility principle:** ogni componente del software deve avere una sola responsabilità, cioè un solo motivo per cambiare, quindi deve svolgere una sola funzionalità
 - si basa sul concetto di coesione, cioè su quanto un componente supporta un'unica responsabilità, ne consegue che nel design di un componente è richiesta un'alta coesione
 - il rispetto di questo principio comporta una migliore manutenibilità del codice, che può essere modificato o aggiornato con un minore impatto sul resto del sistema
- **The open closed principle:** il software deve essere aperto alle estensioni e chiuso rispetto alle modifiche
 - il codice già scritto e funzionante non deve essere modificato, nel caso si vogliano modificare o aggiungere delle funzionalità non si deve impattare sul codice esistente (si devono creare delle nuove classi)
 - alla base dei design pattern che favoriscono la composizione all'ereditarietà
- **The liskov substitution principle:** gli oggetti di una superclasse devono essere rimpiazzabili dagli oggetti di una sua sottoclasse senza rompere l'applicazione
 - se ho una classe B che eredita da una classe A, l'applicazione deve funzionare allo stesso modo sia con oggetti di tipo B che di tipo A
 - esempio: se ho un'interfaccia con diversi metodi e ho tante classi che svolgono una singola funzione, le classi sono costrette a implementare anche metodi al di fuori della singola responsabilità che svolgono
- **The interface segregation principle:** una classe deve dipendere solo dalle interfacce che sono per essa necessarie
 - i metodi di un'interfaccia devono essere coerenti con la sua funzionalità, non devono esserci metodi implementati che non riguardano la classe che li sta implementando
 - esempio: supponiamo di avere un'interfaccia *Atleta* e una classe *Calciatore* che eredita da *Atleta* i metodi *run()*, *jump()* e *swim()*; in questo caso il principio viene violato perché il calciatore non nuota, ma deve lo stesso implementare il metodo *swim()*
- **The dependency inversion principle:** dobbiamo dipendere da astrazioni e non da cose concrete
 - i moduli ad alto livello di astrazione non devono dipendere dai sottomoduli che ne forniscono un'implementazione
 - un cambiamento a basso livello non deve impattare sui moduli di alto livello

*Principi per il design dei componenti: principi relativi alla coesione e all'accoppiamento Misurare la qualità dei design dei componenti

I componenti sono insiemi di entità

Il nostro obiettivo è creare componenti:

- Fortemente coesi, cioè unire le cose che riguardano gli stessi argomenti, che hanno la stessa funzionalità
- Poco accoppiati, cioè poco dipendenti gli uni dagli altri

Principi che regolano la coesione:

- **Release equivalence principle:** durante il rilascio di un componente software assegniamo un numero e un nome alla release, questo consente di capire a quale versione un componente fa riferimento, in modo da poter tracciare le funzionalità dei componenti appartenenti a quella release, ogni release rappresenta insieme un “pacchetto unico” per l'utilizzo
- **Common closure principle:** classi che cambiano per la stessa ragione e nello stesso momento vanno messe nella stessa release; se ad esempio ho 5 classi di cui 2 cambiano frequentemente, non ha senso metterle tutte nello stesso pacchetto perché quando dovrò aggiornare, 3 classi verranno rimesse senza essere modificate
- **Common reuse principle:** non devo forzare gli utenti che usano un componente a dipendere da cose che non usano; consideriamo una release con 5 classi, se 3 di queste vengono usate internamente e solo 2 di queste sono rilevanti per l'utente, stiamo fornendo delle informazioni inutili a chi sta usando il componente

Principi che regolano l'accoppiamento:

- **Acyclic dependencies principle:** i vari pacchetti non devono essere in dipendenza ciclica
- **Stable dependencies principle:** devo dipendere in direzione della stabilità, cioè da qualcosa che varia meno velocemente di me; se sviluppo un componente che dipende da un altro componente che varia frequentemente, sarò sempre ad “inseguire” il cambiamento (a cui sono legata tramite la dipendenza), si deve cercare di dipendere da componenti “stabili”, ovvero che variano poco o meno velocemente rispetto al mio componente
- **Stable abstraction principle:** più un componente deve essere astratto più al suo interno deve contenere classi astratte, per garantire la stabilità di un componente bisogna inserire al suo interno il concetto di astrazione (cioè classi astratte), Astrazione = numero di classi astratte / numero di classi totali

★**Principi clean dell'architettura: indipendenza, disaccoppiamento dei layer / degli use case**

(pdf 5.2, slide 10-11-13-14)

Una buona architettura deve supportare:

- Gli use case del sistema: lo scopo del sistema deve essere visibile anche a livello architetturale
- Il funzionamento del sistema: questa è una delle opzioni che viene lasciata aperta da un buon architetto
- Lo sviluppo del sistema: Conway law = le aziende che progettano un sistema devono produrre design che sono copie delle strutture comunicative delle aziende stesse, un sistema sviluppato da un'azienda con molti team deve avere un'architettura che facilita l'indipendenza delle azioni dei team stessi cosicché i team non interferiscano gli uni con gli altri durante lo sviluppo
- La distribuzione del sistema: l'obiettivo è “distribuire immediatamente”

L'architetto vuole che la struttura del sistema supporti tutti gli use case, ma non li conosce
L'architetto conosce lo scopo del sistema e può seguire il Single Responsibility Principle e il Common Closure Principle

Il sistema dovrebbe essere diviso in livelli orizzontali disaccoppiati

Il concetto di indipendenza è basato sulla separazione di "strati" del software creati sulla base del ruolo che svolgono: layer della UI, layer per i database, layer di business logic

Gli use case vanno disaccoppiati, ogni funzionalità deve essere autonoma rispetto alle altre e rispetto ai layer orizzontali

Questo disaccoppiamento aiuta nella gestione del processo, poiché permette di assegnare a diversi team diversi task (perché sono indipendenti)

Gli use case cambiano per differenti motivi

Gli use case sono un modo naturale per dividere il sistema, sono i livelli verticali che dividono gli strati orizzontali del sistema

Se disaccoppio gli elementi del sistema che cambiano per motivi diversi allora posso continuare ad aggiungere use case senza interferire con quelli già presenti

★Principi clean dell'architettura del software: policy e livelli / entità

Le architetture plug-in sono formate da due tipi di componenti:

- Business rules: regole
- Implementazione dei dettagli delle business rules

Le business rules non sono tutte allo stesso livello, devono essere separate e raggruppate in base a come cambiano

L'obiettivo è tenere alto e basso livello separati e far puntare le dipendenze in direzione dell'alto livello

Le entità sono le regole più generali in assoluto quelle di più alto livello, quelle su cui si basa il software, quelle che non devono cambiare, quelle che definiscono il funzionamento dell'applicazione stessa senza pensare all'implementazione

★Principi clean dell'architettura del software: principali componenti dell'architettura

Entities: oggetti "modello" che servono per rappresentare gli attori in gioco, rappresentano lo strato più interno

Use cases: usano le entità per realizzare delle implementazioni concrete

Controller: tutti gli oggetti che consentono l'attivazione degli use case, le interfacce che permettono di utilizzare le implementazioni

External interface: è lo strato più esterno, garantiscono la visualizzazione e la persistenza degli elementi

★Che cos'è un'architettura del software e quali possono essere degli esempi di stili architetturali?

L'architettura del software è la struttura del software stesso, cioè l'insieme degli elementi di alto livello di cui si compone il software, delle proprietà visibili all'interno del software, delle relazioni fra i componenti e fra componenti e proprietà

Le architetture possono essere:

- Monolitiche: il codice che gestisce tutte le funzionalità del progetto è su una sola macchina
- Distribuite: il progetto è gestito su più macchine in remoto che comunicano tra loro, architettura più elastica perché concepisce il software come funzionante su diversi dispositivi, i server possono essere specializzati a seconda dell'ambito in cui il componente deve essere utilizzato (ad esempio un server che deve gestire gli accessi di tanti utenti sarà più potente di un server che necessita di prestazioni più basiche), questo tipo di architettura è basata sulla rete (influenzata dal tempo di latenza e dalla stabilità della rete)

Degli esempi di stili architetturali possono essere:

- Pipes and filters
- Client server
- Layered
- Microkernel
- Event driven
- Microservizi
- Service based
- Serverless

★**Stili architetturali: pipes and filters, client server, layered, microkernel, event driven, microservizi, service based e serverless**

Pipes and filter

Il sistema è composto da due tipi di componenti:

- Pipes, cioè i collegamenti tra i componenti
- Filters, cioè i componenti che realizzano l'applicazione, possono essere eseguiti in parallelo, i filter possono essere di diversi tipi:
 - Producer
 - Transformer
 - Tester
 - Consumer

Applicazioni molto specifiche di un dominio che funzionano senza l'interazione con l'utente

Ogni componente ha una sua autonomia, ma tutti i componenti sono costruiti verso un obiettivo comune

Architettura facile da mantenere e aggiornare

Client server

Il client è il dispositivo che ha bisogno dell'esecuzione dell'operazione, mentre il server è la macchina che risponde all'esecuzione dell'operazione

Il client può connettersi a più server

Architettura influenzata da cambiamenti e costi dell'hardware

Tempo fa i client non erano così potenti quindi gran parte delle operazioni venivano fatte sul server, ora anche i client possono eseguire operazioni e ciò rende più veloce l'esecuzione

Layered

Le applicazioni sono divise in strati logici orizzontali che riguardano la tecnica di come è stato realizzato il software (e non il contenuto)

Se gli strati sono chiusi possono comunicare solo con quelli a loro adiacenti (agevolano manutenzione e scalabilità), talvolta è conveniente avere strati aperti, cioè che possono comunicare con qualsiasi altro strato

Solitamente in ogni architettura sono presenti quattro strati standard:

- Presentazione
- Regole
- Persistenza
- Dati

Se gli strati sono chiusi possono essere anche isolati, cioè i cambiamenti in uno strato non influenzano gli altri strati

Microkernel

Sistema core a cui sono collegati elementi aggiuntivi detti plug-in

Un registro permette al sistema core di sapere quali plug-in sono disponibili e come ottenerli

I plug-in devono conoscere la funzione che devono svolgere una volta installati

Se dovesse essere implementato con un design pattern sarebbe il command

Event driven

Architettura che risponde a degli eventi

- Architettura a mediatore: il mediatore è un elemento centralizzato che conosce la successione degli eventi, il mediatore è l'elemento debole (deve conoscere tutte le operazioni, se dovessi aggiungere un'operazione non rispetterei molto l'open closed principle)
- Applicazione a broker: ogni componente sa qual è quello successivo nella catena, molto elastica, se voglio aggiungere un elemento devo modificare solo quello che lo chiama

Microservizi

Ogni componente è distribuito come un'unità separata

Ogni servizio ha il suo processo

Il servizio viene riassunto al minimo

Ogni servizio ha il proprio database

Service based

Un server offre delle funzionalità utilizzate da un client

L'applicazione viene suddivisa in funzionalità (blocchi verticali) implementate in maniera autonoma

Condividono un'unica base di dati (debolezza)

Serverless

Applicazione concepita come un insieme di applicazioni

Design characteristics and metrics

*In che modo può essere descritta e caratterizzata la progettazione di un software?

La progettazione di un software mira alla modularizzazione, per applicare i concetti di coesione e indipendenza tra moduli

La progettazione è l'insieme delle attività che, a partire dalla specifica dei requisiti del sistema software, producono un modello del sistema da realizzare

Collega la parte di specifica e la parte di codifica

*Cosa si intende per metrica del software, quali sono i limiti nell'utilizzare metriche per caratterizzare i software e quali tipi di metriche possono essere utilizzate?

Una metrica del software rappresenta una misura quantitativa del grado di cui un sistema software, un componente o un processo possiede una definita proprietà o caratteristica

L'obiettivo delle metriche del software è di fare valutazioni durante tutto il ciclo di vita del software per confermare se la qualità del software incontra o meno i requisiti

L'uso delle metriche del software riduce la soggettività nella valutazione e nel controllo della qualità del software fornendo basi quantitative per prendere decisioni sulla qualità del software
Tuttavia l'uso delle metriche del software non elimina la necessità del giudizio umano nella valutazione del software

L'uso delle metriche del software in un progetto dovrebbe avere un effetto benefico rendendo la qualità del software più visibile

I limiti nell'utilizzo delle metriche sono nel non poter stabilire dei range universali in cui il software rientra, rischiando di adattare il proprio prodotto per soddisfare le metriche stesse

Tipi di metriche che possono essere utilizzati:

- Metriche di prodotto: caratterizzano un prodotto intermedio o finale (esempio: dimensione del codice, complessità...)
- Metriche di processo: caratterizzano metodi, tecniche e strumenti utilizzati per sviluppare, implementare e mantenere il software (esempio: manutenibilità, velocità di risoluzione di bug...)
- Metriche di progetto: caratterizzano il progetto i termini di tempo, costo e management (esempio: tempistiche di progetto, produttività...)

*Cosa si intende per analisi statica e dinamica del software

L'analisi statica è il processo di valutazione di un sistema o di un suo componente basato sulla sua forma, sulla sua struttura, sul suo contenuto o sulla documentazione di riferimento

Ciò significa che la valutazione avviene a prescindere dall'esecuzione del sistema
(Esempio: lines of code, nested level...)

L'analisi dinamica è il processo di valutazione di un sistema software o di un suo componente basato sull'osservazione del suo comportamento in esecuzione
(Esempio: code coverage, bugs per line of code...)

*Caratteristiche delle misure basate

- Sul conteggio delle linee di codice
- Sul livello di innestamento del codice

- **Sulla complessità del codice (Halstead e complessità ciclomatica di Mc Cabe)**

Linee di codice: metriche che misurano quantitativamente la dimensione di una sorgente di codice contando le linee di codice

- per determinare l'ordine di magnitudine dell'implementazione
- misura statistica per stimare e predire lo sforzo
- per stimare i costi generali
- determinare quantitativamente il grado rispetto cui il codice sorgente cambia o si evolve nel tempo
- non considerano la complessità
- dipendono dal linguaggio di programmazione
- può essere applicata a tutte le unità di codice (metodi, classi, package...)
- usata per dare una visione ampia della grandezza del codice

Nesting level(NL): metriche che misurano quantitativamente la complessità di una sorgente di codice misurando la profondità dell'innestamento delle strutture

- per misurare la complessità di un'unità di codice
- identifica le parti di codice difficili da capire o mantenere
- conta a che livello di innestamento si trova il codice che stiamo valutando
- dobbiamo tendere ad avere un livello massimo di NL basso perché è più semplice da gestire e modificare
- controlla le funzioni innestate, le classi innestate, i blocchi di codice innestati come costrutti condizionali o funzioni
- indicatore di quali parti del codice possono essere eleggibili per un refactoring

Halstead complexity: basata su un'analisi statica che viene concentrata sulla complessità lessicografica del codice

Da questa misura si riesce ad estrapolare una metrica di complessità

Gli operatori e gli operandi sono definiti dalle loro relazioni reciproche

Si analizzano gli operatori (coloro che effettuano le azioni) e gli operandi (coloro che partecipano alle azioni)

Gli operatori sono i costrutti mentre gli operandi sono le variabili

Complessità ciclomatica di McCabe: consente di valutare il numero di percorsi che sono linearmente indipendenti in un software, due percorsi sono linearmente indipendenti se differiscono gli uni dagli altri almeno per una variazione

- $\text{complessità} = E - N + 2p$

★ **Quali sono i problemi generati dalla duplicazione del codice**

Ci sono diversi argomenti che sostengono il fatto che la duplicazione del codice aumenti lo sforzo nella manutenzione:

- I cambiamenti devono essere fatti molte volte se il codice è ridondante
- Spesso non è documentato dove il codice è duplicato
- La ricerca manuale del codice duplicato è impossibile in grandi sistemi

- Durante l'analisi, lo stesso codice può essere letto diverse volte, poi comparato con altro codice solo per scoprire che questo codice è già stato analizzato

Se il codice fosse implementato una sola volta in una funzione questo sforzo potrebbe essere evitato

La duplicazione viola uno dei principi fondamentali ovvero il DRY (don't repeat yourself)

*Come si possono misurare le relazioni tra i moduli del software?

Una dipendenza denota una relazione formale tra due moduli

Le dipendenze possono essere rilevate durante l'analisi statica del codice

Un set di moduli è detto clique se per ogni coppia esiste una dipendenza

Le metriche di Henry-Kafura misurano il flusso tra i moduli, sono basate sul flusso di informazioni che entrano ed escono dai moduli

Un flusso di informazioni è una misura dell'interazione tra moduli o tra moduli e ambiente

Fan in: numero di flussi di informazioni in ingresso a un modulo, numero totale di moduli dipendenti dal modulo dato

Fan out: numero di flussi di informazioni in uscita da un modulo, numero totale di moduli da cui il modulo dato dipende

La complessità di un modulo è: $C_p = (fan-in + fan-out)^2$

Le dipendenze possono essere del tipo:

- A chiama B
- A eredita da B
- A usa una variabile di B

*Cosa si intende e come si descrivono la coesione e l'accoppiamento dei moduli di un software?

Per mantenere un design semplice dobbiamo assicurarci che ogni unità del design sia focalizzata su un singolo obiettivo

Coesione: caratteristica di un'unità che identifica il grado a cui gli elementi di questa unità appartengono

In un'unità di design altamente coesa, i suoi componenti sono tutti connessi tra loro per soddisfare un singolo obiettivo o una singola funzione

Diversi livelli di coesione:

- Coesione casuale: l'unità di design o codice sta eseguendo tanti task non correlati, livello di coesione più basso
- Coesione logica: il design esegue una serie di task simili, gli elementi sono correlati ma le relazioni sono deboli
- Coesione temporale: mette insieme una serie di elementi correlati dal tempo, azioni collegate in termini di tempo, in una classe ho funzionalità accedute nello stesso momento nel software
- Coesione procedurale: coinvolge azioni correlate proceduralmente, azioni collegate in termini di sequenza
- Coesione comunicazionale: il design è correlato dalla sequenza delle attività, e le attività sono rivolte agli stessi dati
- Coesione sequenziale o funzionale: le unità di design eseguono una principale attività o raggiungono un obiettivo, una classe svolge una sola funzionalità

Accoppiamento: caratteristica che indirizza il grado dell'interazione e dell'interdipendenza tra due unità del software

Anche se raggiungiamo un altissimo livello di coesione tra le unità del software, queste unità avranno la necessità di interagire tramite il processo di accoppiamento (l'interazione deve essere minima per garantire il riuso del codice e la sua comprensione)

Le classi strettamente accoppiate sono difficili da riutilizzare separatamente

Una classe (o un modulo) che è molto dipendente da altre classi (o moduli) sarà molto difficile da capire

Senza capire tutte le dipendenze tra classi e moduli sarà difficile riusare, modificare o correggere il codice

Se c'è un errore in una classe molto dipendente e strettamente connessa con altre classi, la probabilità di propagazione dell'errore è alta

L'alto accoppiamento non è una caratteristica di design desiderabile

Diversi livelli di accoppiamento:

- Accoppiamento dei contenuti: due unità accedono reciprocamente a dati e informazioni, ogni cambiamento in una unità comporterà un attento controllo dell'altra unità
- Accoppiamento comune: due classi hanno accesso alle stesse variabili globali
- Accoppiamento controllato: un'unità passa il controllo delle informazioni a un'altra unità
- Accoppiamento stamp: le unità di codice passano strutture dati alle altre unità
- Accoppiamento dei dati: le unità di codice passano solo i dati necessari alle altre unità

In generale quando si progetta un software si deve seguire la regola di forte coesione e basso accoppiamento

*Quali sono le principali misure che caratterizzano un software costruito con un linguaggio Object Oriented (CK metrics)

Le CK metrics misurano la complessità del design in relazione all'impatto sulla qualità degli attributi esterni come manutenibilità, riuso, validazione

- Peso dei metodi nelle classi: somma pesata di tutti i metodi di una classe
- Profondità dell'albero di ereditarietà: lunghezza massima di ereditarietà di una classe a partire dalla sua radice
- Numero di figli: numero di figli di una classe, o le sue sottoclassi
- Accoppiamento tra gli oggetti nelle classi: numero di classi a cui una classe è accoppiata, numero di classi da cui il modulo dipende
- Risposta di una classe: set di metodi che verranno coinvolti nella risposta a un messaggio (mandato per un oggetto di quella classe)
- Mancanza di coesione nei metodi: misura la correlazione tra i metodi e le istanze di variabili locali di una classe

Implementation

*Che cosa si intende per fase di implementazione e quali sono le caratteristiche di questa fase?

In che modo le caratteristiche si influenzano e quando assumono una particolare importanza? Che importanza è opportuno dare all'ottimizzazione del codice?

L'implementazione è la fase che trasforma i requisiti e i concetti del design in codice con un determinato linguaggio di programmazione

Le caratteristiche dell'implementazione sono:

- Leggibilità: quanto il codice è comprensibile per chi lo legge (poco innestato, pochi commenti)
- Manutenibilità: facilmente modificabile
- Performance: il codice deve lavorare il più veloce possibile
- Tracciabilità: il codice deve seguire un certo elemento di progettazione o design
- Correttezza: l'implementazione svolge la funzione per cui è nato il software (valutazione e validazione)
- Completezza: tutti i requisiti sono soddisfatti

*Nozione dello standard ISO/EIC 25010

Lo standard ISO/EIC 25010 valuta la qualità del lavoro svolto e propone 2 modelli:

- Software product quality model: le caratteristiche di qualità che derivano dal mercato possono essere misurate internamente o esternamente, valutiamo il software senza vederlo funzionare
 - Idoneità funzionale: grado secondo cui un prodotto impiega risorse quando è usato sotto specifiche condizioni, quanto si avvicina alla richiesta del cliente
 - Efficienza delle performance: numero di risorse utilizzate
 - Compatibilità: grado secondo cui due o più sistemi o componenti possono scambiare informazioni
 - Usabilità: grado secondo cui un sistema è utilizzato da specifici utenti per ottenere specifici obiettivi
 - Affidabilità: grado secondo cui un sistema o un componente esegue specifiche funzioni sotto specifiche condizioni per uno specifico periodo di tempo, numero di crash del sistema
 - Sicurezza: grado secondo cui un prodotto o sistema protegge informazioni e dati
 - Manutenibilità: grado secondo cui un prodotto può essere modificato, quanto è facile aggiornare il software
 - Portabilità: grado secondo cui un prodotto può essere trasferito, se può essere eseguito su diverse piattaforme
 - internamente: misura attributi staticamente ispezionando e analizzando il software, white box overview, utilizzato durante lo sviluppo
 - esternamente: misura gli attributi dinamicamente eseguendo il software, black box overview
- Quality in use model: le caratteristiche sono misurate quando il software è testato in un contesto d'uso realistico, valutiamo il software quando in uso
 - Efficacia: accuratezza e completezza con cui gli utenti raggiungono obiettivi specifici, quanto impiega ad eseguire il suo compito
 - Efficienza: le risorse spese in relazione all'accuratezza e completezza con cui gli utenti raggiungono obiettivi specifici, se fa quello per cui è stato creato
 - Soddisfazione: il grado secondo cui le necessità degli utenti sono soddisfatte, quanto il cliente è soddisfatto del suo utilizzo

- Libertà dai rischi: il grado secondo cui un prodotto o un sistema mitiga le potenzialità dei rischi, quanto il software può impattare sul pericolo che può causare
- Copertura nel contesto: il grado secondo cui un prodotto un sistema può essere usato con efficacia, efficienza, libertà dai rischi, soddisfazione

★Cosa si intende per stile di programmazione?

Per stile di programmazione si intende seguire delle linee guida specifiche per la scrittura del codice, che lo rendano consistente, leggibile e manutenibile

Esempi:

- scegliere buoni nomi seguendo le stesse regole: migliora la consistenza, la leggibilità e la manutenibilità
- separare le parole
- indentazione e spazi
- dimensione di funzioni e metodi: ad esempio solo un certo numero di linee può essere impiegato per implementare una funzione o un metodo, è provato che in metodi o funzioni di grandi dimensioni è più alta la presenza di errori
- nomi dei file
- diversi linguaggi di programmazione supportano diverse caratteristiche: usare costrutti safe
- consistenza del codice: a volte le aziende possono imporre alcune regole come ad esempio l'utilizzo dell'ereditarietà multipla o la compilazione senza warnings

★Come possono essere classificati i commenti?

I commenti possono aiutare o ledere la leggibilità e la manutenibilità del codice

Essi possono: distogliere l'attenzione dal codice, rendere il programma più difficile da leggere, essere errati, inoltre devono essere aggiornati ogni volta che viene aggiornato il codice

Ci sono 6 tipi di commenti diversi:

- Ripetizione del codice: commenti su istruzioni banali, questi commenti devono essere evitati perché sono inutili
- Spiegazione del codice: commento che spiega quello che è stato scritto tramite il codice, il codice dovrebbe essere comprensibile senza spiegazione, se così non fosse allora dovrebbe essere riscritto
- Indicatori nel codice: indicazioni sotto forma di commento che servono per capire cosa fare in futuro o cosa ricontrollare, indicano oggetti incompleti, miglioramenti, tengono traccia dei cambiamenti (commenti corretti)
- Riassunto del codice: commenti che riassumono ciò che fa il codice, sono utili per capire il codice ma vanno tenuti aggiornati (commento corretto, ma evitabile)
- Descrivere l'intenzione del codice: descrivono cosa il codice dovrebbe fare e non cosa fa, se seguiamo l'open closed principle allora l'intenzione del codice non cambierà mai (quindi commento corretto)
- Riferimenti esterni: commenti che collegano il codice a entità esterne, mettere riferimenti a quello che abbiamo fatto ("Ho implementato l'algoritmo X")

★Cosa si intende per debugging e quali sono le fasi di quel processo?

Il debugging è l'atto di localizzare e correggere gli errori nel codice, errori che sono stati precedentemente intercettati tramite il testing

Il debugging è un processo iterativo in cui: si crea un'ipotesi sulla causa dell'errore, si scrivono dei test per provare l'ipotesi e poi si cambia il codice per provare a risolvere il problema

Il debugging si divide in 4 fasi:

- Stabilizzazione o riproduzione: si riproduce l'errore su una configurazione per capire le condizioni che ci hanno condotto all'errore costruendo un test case
- Localizzazione: si trova la sezione di codice che provoca l'errore
- Correzione: si cambia il codice per risolvere l'errore
- Verifica: si verifica che gli errori siano stati corretti e che non siano stati introdotti nuovi errori nel codice

★Cosa si intende per programmare per asserzioni?

In che modo Java supporta quel processo?

Quali sono le funzionalità offerte da junit per il debugging del software?

Le asserzioni sono statement che verificano una condizione e producono un errore se la condizione non viene soddisfatta

Quando si programma per asserzioni si impongono delle istruzioni per far proseguire il codice in un certo modo

Ad esempio, possiamo imporre che in un punto del programma una variabile debba avere un certo range di valori, settiamo un'asserzione in quel range

Quando durante il testing eseguiamo il software, se la condizione imposta non viene verificata il programma si blocca

Le condizioni che si possono verificare con le asserzioni sono:

- Precondizione: condizione che un modulo richiede per produrre un risultato corretto
- Postcondizione: condizione che dovrebbe essere vera dopo aver eseguito il codice
- Loop invariants: modalità per capire cosa fa un loop

★Cosa si intende per defensive programming?

È una pratica secondo cui gli sviluppatori anticipano i fallimenti del codice

Opera sulla prerogativa che nel nostro codice esistano dei bug, quindi dobbiamo trovare il modo più facile per identificarli o contrastarli

Consigli che aiutano a scrivere il codice in modo tale che ci sia meno probabilità di scrivere errori

★Cosa si intende per refactoring del codice?

Il refactoring consiste nell'attività di miglioramento dello stile del codice senza alterarne il comportamento

Il codice può essere migliorato se:

- c'è codice duplicato
- i metodi sono lunghi
- le classi sono grandi
- abbiamo molti switch
- un metodo tende ad usare un oggetto di una classe diversa da quella da cui proviene
- una classe si riferisce troppo a parti private di altre classi

Il refactoring è un cambiamento nella struttura interna del software che lo rende più facile da capire e da modificare senza cambiarne il comportamento

Tipi di refactoring:

- extract method: processo che trasforma un frammento di codice in un proprio metodo
- substitute algorithm: un processo che sostituisce il corpo di un metodo con un nuovo algoritmo più chiaro e ritorna lo stesso risultato
- move method: un processo che sposta un algoritmo da una classe all'altra in cui ha più senso
- extract class: un processo che divide una classe in più classi

*Indicare le principali linee guida per lo sviluppo del codice

1. Scrivere unità di codice corte: limitare la lunghezza delle unità di codice a 15 linee, il codice con una singola responsabilità è più facile da testare, analizzare e riutilizzare
 - Tecniche di refactoring: extract method, replace method with method object
2. Scrivere unità di codice semplici: limitare i punti di ramificazione a 4 per ogni unità, $NL_{max}=4$
 - Tecniche di refactoring: extract method, replace conditional con polimorfismo, replace conditional con clausole di guardia
3. Scrivere il codice una volta sola: non copiare il codice, quando il codice viene copiato è necessario risolvere i banchi in diversi punti e questo rende il codice poco manutenibile
 - Tecniche di refactoring: extract method, extract superclass
4. Mantenere le interfacce piccole: limitare il numero di parametri per unità a 4
5. Separare i problemi in moduli: evitare larghi moduli per ottenere un accoppiamento lasco tra essi, si fa assegnando responsabilità a moduli separati e nascondendo l'implementazione dei dettagli dietro le interfacce
6. Accoppiare lascamente i componenti architetturali
7. Mantenere i componenti architetturali bilanciati: organizzare il codice in modo che il numero di componenti sia circa 9 e che i componenti stessi abbiano circa la stessa dimensione
8. Mantenere la propria codebase piccola: una codebase è una collezione di codice immagazzinata in un repository, può essere compilata e sviluppata indipendentemente, e mantenuta da un team
9. Automatizzare lo sviluppo di pipeline e test
10. Scrivere codice pulito: ad esempio non lasciare commenti inutili o codice inutilizzato

Testing and Quality Assurance

*Cosa si intende per qualità del software e come si ottiene un software di qualità?

In che modo / attraverso quali prospettive si può valutare la qualità (cosa si intende per verifica e per validazione)?

Per creare un software che abbia meno difetti possibili dobbiamo:

- Cercare di attuare strategie per evitare di commettere errori
- Controllare che effettivamente il software abbia le qualità che ci si aspetta una volta rilasciato

La qualità di qualsiasi prodotto può essere definita in due modi:

- Esso è conforme alle specifiche
- Esso serve i propri obiettivi

La verifica è l'atto di controllare che un prodotto software sia conforme ai suoi requisiti e alle sue specifiche

Si assicura che il sistema software incontri tutte le funzionalità

Viene attuata prima

Fatta dagli sviluppatori

Riguarda attività statiche

È un processo oggettivo

La validazione è l'atto di controllare che un prodotto software finito incontri i requisiti e le specifiche dell'utente

Si assicura che le funzionalità incontrino i comportamenti attesi

Avviene dopo la verifica e coinvolge il controllo del prodotto nella sua interezza

Fatta dai tester

Riguarda attività dinamiche, include l'esecuzione del software rispetto i requisiti

È un processo soggettivo

*Come si possono individuare gli errori in un software o in un artefatto a esso connesso?

Tecniche per identificare gli errori:

- testing: processo di progettazione dei test, eseguire il programma in un ambiente controllato, e verificare che l'output sia corretto
- ispezioni e revisioni
- metodi formali: tecniche matematiche usate per provare che un programma sia corretto
- analisi statica: processo di analisi delle strutture statiche di un programma o di un prodotto software intermedio, solitamente automatizzata

*Differenza tra fallimento, difetto e errore

Un difetto è una condizione che può causare un fallimento nel sistema

Può essere causato da un errore fatto dall'ingegnere del software

È anche chiamato bug

Non tutti gli errori sono difetti, potrebbero esserci errori che non impediscono il funzionamento del software

Un fallimento è l'incapacità di un sistema di svolgere una funzione rispetto le sue specifiche

È il risultato di un difetto nel sistema

Il fallimento corrisponde a un difetto emerso che provoca un crash del software

Se un fallimento non emerge con alta probabilità non si sa nemmeno della presenza del difetto alla sua base

Un errore è uno sbaglio fatto dall'ingegnere del software o dal programmatore

Un errore conduce a un difetto che può causare un fallimento

Durante il testing vengono osservati i fallimenti che rivelano la presenza di difetti

***Cosa si intende per testing del software?**

Quali sono gli scopi del testing?

In che modo è possibile fare del test?

Esistono delle procedure automatizzate per supportare il testing?

Definizioni di testing:

- Processo di realizzazione di un software per far emergere differenze tra le condizioni esistenti e quelle richieste e valutare le caratteristiche di un software
- Attività fatta per valutare la qualità del software per migliorarlo identificando difetti e problemi
- Esame sistematico e metodico di un prodotto usando una varietà di tecniche con l'intenzione di mostrare che non soddisfatti gli obiettivi desiderati dagli utenti, fatto in un ambiente che sia più vicino possibile a quello dell'ambiente di produzione

Gli scopi del testing sono:

- Trovare difetti nel software così che possano essere corretti e mitigati
- Fornire una valutazione generale della qualità, che include fornire la sicurezza che il prodotto lavori in molti casi, e una stima dei rimanenti difetti

Il testing non può provare che un prodotto funzioni, può trovare difetti e mostrare che il prodotto lavora nei casi che sono stati testati, senza garantire nulla per gli altri casi

Non si testa solo il codice, ma anche i documenti a margine (documentazione, guide...)

Solitamente il testing viene eseguito in un ambiente produttivo che simuli il più possibile la condizione in cui il software verrà utilizzato

***Chi esegue il testing?**

Quando si fa il testing?

Cosa si intende per testing funzionale e non funzionale?

Il testing viene eseguito da:

- Programmatori: creano i test e li eseguono per convincersi che il programma funzioni
- Tester: lavoratori tecnici che devono scrivere test e garantire la loro esecuzione
- Utenti: aiutano a identificare problemi di usabilità e a esporre il software a un range di input in scenari reali, usa il programma in versione beta sapendo che potrebbe avere dei problemi

Il testing va iniziato dall'inizio della fase di raccolta dei requisiti e continua fino alla distribuzione del software

Il testing può essere

- Statico: si guardano i documenti
- Dinamico: si testa il software in esecuzione

Il testing funzionale è basato sulle specifiche del software che devono essere testate

Controlla che le funzioni vengano implementate correttamente

Guarda alle specifiche funzionalità del sistema
Può essere attuato a tutti i livelli
È anche chiamato black-box testing
Può essere misurato in termini di percentuale di requisiti coperti dal test

Il testing non funzionale prevede il testare un software dai requisiti
Si concentra sugli aspetti architetturali del sistema e su come è stato creato il software (portabilità, manutenibilità, compatibilità, sicurezza...)
Può essere attuato a tutti i livelli
Spesso richiede conoscenze specifiche e abilità

★**Che cosa si intende per black / white box testing (equivalence partitioning, boundary value analysis)**

Black box testing

Tecnica di testing che non ha conoscenza del lavoro interno all'applicazione
Il programmatore simula di essere un utente, non guardiamo il codice, ma l'esecuzione dell'applicazione
Analisi del software senza consultare il codice

Vantaggi:

- efficiente per grandi segmenti di codice
- non richiesto l'accesso al codice
- separa la prospettiva utente dalla prospettiva sviluppatore tramite ruoli visibili
- i tester possono testare l'applicazione senza conoscenza dell'implementazione
- non necessita della presenza di programmatori esperti

Svantaggi:

- copertura limitata, è testato solo un numero limitato di scenari
- inefficiente, il tester ha solo una conoscenza limitata dell'applicazione
- copertura cieca, il tester non può identificare le aree più propense ad avere errori, non sappiamo quale pezzo di codice stiamo testando
- i test case sono difficili da progettare

Quando si utilizzano le tecniche di black box testing si può fare riferimento a due metodi principali che indicano quali valori è necessario testare, questi metodi sono: il partizionamento delle classi di equivalenza, e l'analisi dei valori limite

Il partizionamento delle classi di equivalenza è basato sul dividere l'input in diverse classi che sono equivalenti poiché hanno l'obiettivo di trovare errori

Si impongono delle partizioni in classi di equivalenza, si dividono gli input per poterli testare

Se un programma fallisce (è corretto) per un membro di una classe, ci si aspetta che fallisca (sia corretto) per tutti i membri di quella classe

Le classi di equivalenza sono determinate guardando la specifica dei requisiti e l'intuizione del tester, senza guardare l'implementazione

L'unione delle classi di equivalenza deve coprire tutto il dominio

In nessun punto due classi devono sovrapporsi

Classi:

- valide: descrivono situazioni che il sistema dovrebbe gestire normalmente
- invalide: descrivono situazioni invalide che il sistema dovrebbe rifiutare

Non combinare classi invalide in un singolo test perchè un valore invalido potrebbe mascherare l'incorretta gestione di un altro valore invalido
Il partizionamento delle classi di equivalenza deve essere disgiunto
Nessuna delle partizioni di equivalenza deve essere vuota
L'unione delle partizioni deve dare il set originale da cui discende la partizione

L'analisi dei valori limite usa il partizionamento delle classi di equivalenza testando i confini piuttosto che gli elementi della classe

Un'analisi completa dei limiti testa il caso immediatamente incluso nel limite e il caso immediatamente escluso

Questa tecnica può generare molti test case, che si possono ridurre solo testando le classi appena sopra e sotto i limiti

Si testa un intorno del valore di cambiamento

Due modalità:

- test con 3 valori: massimo, minimo, valore nominale
- test con 5 valori: massimo, massimo - 1, minimo, minimo + 1, valore nominale

White box testing

Controllo dettagliato della logica interna e della struttura del codice

Il tester necessita di guardare il codice sorgente e trovare che unità di codice si sta comportando in modo inappropriato

Analisi del software con consultazione dell'implementazione del codice

Vantaggi:

- il tester conosce il codice sorgente quindi è facile identificare che tipi di dati aiutano nel testing
- aiuta a ottimizzare il codice
- linee di codice extra possono essere rimosse, il che può portare a difetti nascosti
- massima copertura del codice perchè il tester conosce il codice

Svantaggi:

- il tester deve essere specializzato quindi i costi aumentano
- a volte è impossibile guardare ogni angolo del codice per trovare errori nascosti
- difficile da mantenere perchè necessita di strumenti specifici

★Che cosa sono e a cosa servono le decision table?

Le decision table sono un metodo per riconoscere quali valori testare quando si utilizza la tecnica del black box testing

Le decision table ci permettono di testare in che modo varie combinazioni di condizioni interagiscono e producono un certo risultato possibile e non possibile

Le tabelle sono costruite con le combinazioni delle condizioni nelle colonne, le condizioni sono in alto, mentre le azioni ad esse associate sono in basso

Esprimono le regole che governano la gestione delle situazioni transizionali

Tabella che contiene tutti i casi possibili e ci permette di compiere un test esaustivo

Esistono dei modi per attuare delle semplificazioni e ridurre il numero di casi possibili eliminando quelli ridondanti

★È possibile generare test a partire da use case?

Si, è possibile generare test a partire da use case

Gli use case diagram rappresentano i requisiti dell'applicazione che stiamo testando, quindi ci indicano cosa deve fare l'applicazione, questo è proprio l'obiettivo del black box testing

Modalità per assicurarci che abbiamo testato scenari tipici e eccezionali per il sistema, dal punto di vista dei vari attori che interagiscono direttamente con il sistema e dal punto di vista dei vari stakeholder che interagiscono indirettamente con il sistema

Gli use case rappresentano i requisiti quindi conoscendo questo diagram possiamo identificare il domino del sistema

Gli use case rappresentano sia i casi validi che i casi particolari (extend)

★Cosa si intende per control flow graph?

Il control flow testing è un approccio al testing basato sulla struttura, in cui i test case sono progettati per eseguire specifiche sequenze di eventi

Il control flow testing è attuato tramite i control flow graph, cioè dei grafi che forniscono un modo per astrarre un modulo di codice per capire meglio cosa fa

I grafi ci danno una rappresentazione visiva della struttura del codice

L'algoritmo consiste nel convertire una sezione del codice in un grafo e analizzare i possibili percorsi tramite il grafo

★Che cosa si intende per statement coverage, decision coverage, loop coverage, condition coverage e modified condition / decision coverage

Statement coverage

Tecnica che valuta le linee di codice

Tecnica di controllo di flusso meno efficiente

Minimo livello di copertura che dovrebbe essere accettabile

Se non riusciamo a testare tutto il codice e dobbiamo lasciare indietro qualcosa, conviene testare i percorsi meno probabili, perché quelli più probabili verranno testati quasi sicuramente da qualcun altro

Gli errori logici sono inversamente proporzionali alla probabilità dell'esecuzione di un percorso

Quando utilizziamo lo statement coverage dobbiamo stare attenti perché a volte con un coverage del 100% (cioè abbiamo testato tutte le linee di codice) non riusciamo effettivamente a testare tutti i casi possibili

Decision coverage

Tecnica che valuta le decisioni, ogni decisione ha la possibilità di essere risolta come vera o falsa

Se ho un decision coverage del 100% ho anche statement coverage del 100%

Loop coverage

Tecnica che testa i cicli

Per testare un ciclo si possono prendere come riferimento i seguenti valori:

- un valore che è 1 meno il minimo valore che il ciclo può prendere
- minimo numero di iterazioni
- 1 più del valore minimo
- Fare il ciclo una volta
- Fare il ciclo 2 volte
- Valore nominale, cioè tipico

- 1 meno del valore massimo
- Massimo numero di valori
- 1 più del valore massimo

Condition coverage

Tecnica che valuta le decisioni non atomiche, quindi le decisioni composte da condizioni

Testo il caso true e il caso false per ogni condizione

Una condizione atomica non può essere decomposta e non può contenere condizioni legate da operatori logici

Il condition coverage non implica il decision coverage, quindi facendo il primo potremmo non rispettare il secondo

Modified condition / decision coverage

Tecnica che guarda l'impatto della condizione nella decisione

Ogni condizione atomica deve essere testata in entrambi i modi e il decision coverage deve essere rispettato

Per ogni condizione atomica c'è almeno un test case in cui l'intero predicato viene valutato come falso solo perché quella specifica condizione atomica è falsa

Per ogni condizione atomica c'è almeno un test case in cui l'intero predicato viene valutato come vero solo perché quella specifica condizione atomica è vera

(esempio: la decisione deve essere vera perché A è vera, a causa di A)

Utile per chi sta testando software critici, ma non esaustivo in qualsiasi situazione

Un valore neutrale per una condizione atomica è un valore che non ha effetto sul risultato dell'espressione, ad esempio per l'and il valore neutrale è il true, mentre per l'or è il false

Garantisce un alto livello di sicurezza

★Che cos'è il path testing?

Il path testing è una tecnica di testing che indentifica i percorsi interessanti nel codice che vogliamo testare

Il primo step di questa tecnica di copertura è di creare un diagramma di controllo di flusso per il codice che vogliamo testare, questo diagramma ci mostra i possibili percorsi percorribili

McCabe teorizzò che ogni modulo software ha un piccolo numero di percorsi unici e indipendenti in esso, questi sono chiamati percorsi base

La sua teoria suggerisce che la struttura del codice può essere testata tramite questo piccolo numero di percorsi e tutti gli infiniti percorsi diversi consistono semplicemente nell'uso e riuso dei percorsi base

L'analisi dei percorsi fornisce una tecnica di progettazione del test che sia riproducibile, tracciabile e numerabile, spesso è usata come una tecnica di tipo white box

Gli obiettivi dell'analisi dei percorsi sono:

- Analizzare il numero di percorsi che esistono nel sistema o nel programma
- Determinare quanti percorsi dovrebbero essere inclusi nel test

Bisogna testare i percorsi linearmente indipendenti, un percorso linearmente indipendente è ogni percorso che introduce almeno un nuovo ramo che non è incluso da nessun altro percorso linearmente indipendente

Due percorsi sono linearmente indipendenti quando uno differisce dall'altro per una sola decisione

Per utilizzare questa tecnica si sceglie il percorso che si pensa essere quello maggiormente seguito e si fa il test su quello, poi si cambia una decisione per volta e si testano tutti i percorsi che derivano dal cambiamento delle decisioni fino a esaurimento

★ Altre tecniche di test: ispezione e review

Esistono 2 tipi di revisione:

- formale: partecipazione di team, documentazione dei risultati, utilizzo di procedure e ruoli definiti (controllo il codice con una checklist e valuto il lavoro fatto)
- informale: non segue un processo definito e non ha come output un documento formale (un collega legge il codice di un altro collega)

La review solitamente è svolta manualmente

Questo lavoro ha anche una funzione di insegnamento nei confronti di chi attua la review e di chi è sotto review

Il processo di revisione è suddiviso in:

- i documenti da revisionare sono studiati dal reviewer
- i reviewer identificano i problemi e informano l'autore in modo formale (cioè con documenti scritti) o in modo verbale
- l'autore decide come agire e aggiorna il documento

Questo processo è sempre presente, in alcuni casi può essere più elaborato

Tipi di review:

- informale: ha l'obiettivo principale di individuare potenziali difetti
- walkthrough: ha l'obiettivo principale di trovare difetti, migliorare il prodotto software, considerare implementazioni alternative, valutare la conformità a standard e specifiche
- tecnica: ha l'obiettivo principale di raccogliere consensi e individuare potenziali difetti
- ispezione: ha l'obiettivo principale di individuare potenziali difetti, valutare la qualità e instaurare fiducia nel prodotto, prevenire difetti simili in futuro migliorando la conoscenza dell'autore (più formale)
- ad hoc: stabilisco una metodologia a seconda del progetto, dipende dalle abilità del reviewer, meno formale
- checklist based: usa una lista per tenere traccia di tutti i controlli da effettuare
- scenarios and dry runs: lo scenario permette delle corse a vuoto del prodotto basate sul comportamento atteso dal prodotto, lo scenario fornisce linee guida migliori per identificare i difetti; guardo come il software funziona in determinate condizioni anche solo simulandolo sulla carta, uso il software in condizioni particolari
- role based: gli artefatti sono esaminati dalla prospettiva di un ruolo specifico (amministratore utenti, amministratore sistema...), faccio controlli simulando di essere un particolare attore del software
- perspective base: il reviewer considera diversi punti di vista degli stakeholder

Tecniche automatizzate: perchè sono importanti?

Per l'esecuzione dei casi di test vengono utilizzati degli appositi framework per automatizzare il testing di unità, ma generalizzabili anche alle problematiche di testing black-box. Questa scelta rappresenta la soluzione più efficace in quanto è possibile monitorare eventualmente anche lo stato interno del software. Il vincolo imposto da questi framework riguarda il linguaggio con cui deve essere scritto il software, che deve supportare la reflection. Un esempio è JUnit che utilizza come linguaggio Java.

Configuration Management, Integration and Builds

***Cosa si intende per software configuration management?**

Da che cosa dipendono gli strumenti software utilizzati a supporto di questa fase?

Software configuration management è il processo di gestione di tutti i pezzi e parti di artefatti prodotti come parte delle attività di sviluppo e supporto software

Strumenti messi a disposizione di chi sviluppa un progetto per supportare, tracciare e gestire al meglio il progetto stesso, un esempio è Git

Il processo di sviluppo del software determina le parti da gestire

Il processo scelto e gli output sviluppati giocano un grande ruolo nella determinazione di cosa dobbiamo gestire e a che livello di dettaglio

Un'organizzazione conservativa e avversa ai rischi che sviluppa un grande software complesso potrebbe avere un processo waterfall con ispezioni che vengono effettuate a seguito di ogni maggiore attività, come la specifica dei requisiti, la specifica di design, la scrittura del codice, la generazione dei test

Un'organizzazione piccola e orientata a processi di tipo agile potrebbe porre enfasi su poche attività, come specifica dei requisiti e codice, potrebbe controllare solo la versione finale di queste attività

***Quali sono le funzioni fondamentali che supportano lo storage e la fase di integrazione e build?**

Il modello di storage e accesso definisce i mezzi necessari per archiviare e controllare l'accesso di tutti gli artefatti software

Alcune funzioni fondamentali sono:

- create: crea un repository
- delete: elimina un repository
- view: consulta la versione attuale e quelle precedenti
- modify: funzione di modifica, deve prevenire i conflitti
- return: per archiviare gli artefatti modificati

La funzione di locking viene utilizzata quando uno sviluppatore blocca una porzione di codice per non creare conflitti

Per quanto riguarda l'integrazione e la compilazione esistono dei tool per segnalare le dipendenze e le corrette compilazioni come Maven o Gradle

Il processo di build è un set di attività associate con l'integrazione e la conversione di file sorgente in un set di file eseguibili rivolti a uno specifico ambiente di esecuzione

Ha due step principali:

- compile: ogni errore di compilazione è corretto e risottomesso per ricompilare
- link: il software è connesso con tutti gli oggetti esterni necessari

Un file speciale è usato per gestire e controllare le attività di build, nei sistemi UNIX è detto makefile

Software Support and Maintenance

Perché la fase di supporto del software e manutenzione è importante?

Quali sono le operazioni che vengono svolte in questa fase?

Dopo che il software viene rilasciato al cliente è fondamentale fornire supporto per la sua manutenzione

Quando il cliente inizierà ad utilizzare il software troverà degli errori e in quel momento si metterà in contatto con l'assistenza fornita da chi ha creato il software per cercare di risolverli

Un software dopo essere stato sviluppato va lentamente verso la sua caduta fino ad essere sostituito, ma questo processo può durare molti anni, in questi anni è fondamentale aggiornare e mantenere il software, garantire questo tipo di assistenza ci assicura anche un riscontro economico positivo

Le operazioni svolte in questa fase sono il contatto con cliente, il quale segnalerà gli eventuali errori, e la manutenzione vera e propria del software, di cui si occupa un apposito team istituito dall'azienda che rilascia il software

È importante stimare il tasso di notifica di problemi?

Sì, e serve per raffigurare una curva dei problemi di arrivo

Un'unità di tempo utilizzato è spesso definita come:

Utilizzo mensile = (utenti che utilizzano attivamente il software) x (mesi di utilizzo)

È vitale stimare accuratamente il problema del tasso di arrivo

Il personale di supporto deve essere formato per quanto riguarda il prodotto software

La preparazione del personale deve iniziare prima che il prodotto sia rilasciato

I dati dei primi mesi sono utilizzati per scovare e tracciare i difetti del software

In che modo può essere strutturato il centro di supporto?

Quali sono i ruoli che svolge?

Quando un utente trova un problema chiama il centro di supporto

Il centro di supporto deve eseguire le seguenti attività:

- identificare l'utente
- ascoltare la descrizione del problema
- verificare che nel database delle FAQ non ci sia già la risposta al problema
- se la risposta non c'è deve fornire una data per eseguire la manutenzione
- fornire un modo per aggirare il problema se possibile
- comunicare il problema al team tecnico di manutenzione

Il centro di supporto prevede:

- supporto ai clienti: risolve solo i problemi che non richiedono manutenzione del codice
- chiamata automatica: aiuta i supervisori a collezionare informazioni sul management